

1 Sorted Runtimes

We want to sort an array of N **unique** numbers in ascending order. Determine the best case and worst case runtimes of the following sorts:

- (a) Once the runs in merge sort are of *size* $\leq N/100$, we perform insertion sort on them.

Best Case: $\Theta(N)$, Worst Case: $\Theta(N^2)$

- (b) We can only swap adjacent elements in selection sort.

Best Case: $\Theta(N^2)$, Worst Case: $\Theta(N^2)$

- (c) We use a linear time median finding algorithm to select the pivot in quicksort.

Best Case: $\Theta(N \log N)$, Worst Case: $\Theta(N \log N)$

- (d) We implement heapsort with a min-heap instead of a max-heap. You may modify heapsort but must maintain constant space complexity.

Best Case: $\Theta(N \log N)$, Worst Case: $\Theta(N \log N)$

- (e) We run an optimal sorting algorithm of our choosing knowing:

- There are at most N inversions

Best Case: $\Theta(N)$, Worst Case: $\Theta(N)$

- There is exactly 1 inversion

Best Case: $\Theta(1)$, Worst Case: $\Theta(N)$

- There are exactly $(N^2 - N)/2$ inversions

Best Case: $\Theta(N)$, Worst Case: $\Theta(N)$

totally reversed

2 MSD Radix Sort

Recursively implement the method `msd` below, which runs MSD radix sort on a List of Strings and returns a sorted List of Strings. For simplicity, assume that each string is of the same length. You may not need all of the lines below.

In lecture, recall that we used counting sort as the subroutine for MSD radix sort, but any sort works! For the subroutine here, you may use the `stableSort` method, which sorts the given list of strings in place, comparing two strings by the given index. Finally, you may find following methods of the `List` class helpful:

1. `List<E> subList(int fromIndex, int toIndex)`. Returns the portion of this list between the specified `fromIndex`, inclusive, and `toIndex`, exclusive.
2. `addAll(Collection<? extends E> c)`. Appends all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's iterator.

```

1  public static List<String> msd(List<String> items) {
2
3      return msd(items, 0);
4  }
5
6  private static List<String> msd(List<String> items, int index) {
7
8      if (items[0].length() ≤ index || items.size() ≤ 1) {
9          return items;
10     }
11     List<String> answer = new ArrayList<>();
12     int start = 0;
13     stableSort(items, index);
14     for (int end = 1; end ≤ items.size(); end += 1) {
15
16         if (items[end].charAt(index) != items[start].charAt(index) // end == items.size()) {
17
18             start = end;
19
20
21     }
22     List<String> sub = msd(items.subList(start, end), index+1);
23     answer.addAll(sub);
24 }
25
26 return answer;
27 }
28
29 /* You don't need to understand the implementation of this method to use it! */
30 private static void stableSort(List<String> items, int index) {
31     items.sort(Comparator.comparingInt(o -> o.charAt(index)));
32 }

```

3 Shuffled Exams

For this problem, we will be working with Exam and Student objects, both of which have only one attribute: sid, which is a number like any student ID.

PrairieLearn thought it was ready for the final. It had meticulously created two arrays, one of Exams and the other of Students, and ordered both on sid such that the ith Exam in the Exams array has the same sid as the ith Student in the Students array. Note the arrays are not necessarily sorted by sid. However, PrairieLearn crashed, and the Students array was shuffled, but the Exams array somehow remained untouched.

Time is precious, so you must design a $O(N)$ time algorithm to reorder the Students array appropriately **without** changing the Exams array!

Hint: Begin by reordering **both** the Students and Exams arrays such that ith Exam in the Exams array has the same sid as the ith Student in the Students array.

1. Sort both by Radix sort. sid has fixed length, and it's number.

So. w is width of sid. $R=10$, $N = \text{count of students/exams}$.

$$O(WR + WN) \approx O(N).$$

2. Match sid to exams and students.

3. Reorder students array based on original exam array.